# UMD package 2.11 — quick help

The scripts in the order of the workflow and of the panels of the flowchart (`documentation/umd-flowchart.pdf`). <base> = the UMD file without `.umd.dat`. All the options of a script: `-h`. `-f all`: every `*.umd.dat` of the folder, one after the other. Run them from `src/` after compiling the C libraries (`README.txt`).

**`umd_workflow.py`** — runs all the analyses of the panels below on UMD files, check first, and summarizes the results; a rerun continues where it stopped and redoes only the steps whose options changed.

```
umd_workflow.py -f a.umd.dat,b.umd.dat [-g structural] [-t transport] [-d thermodynamics] (none = all)
-i 1000 -s 1 -c Si,Mg -a O -o
```

**out:** all the files of the scripts, `umd_workflow.summary.dat` (one line per run: P, T, ρ, E, Cv, D, η, $S_{2PT}$, CN, species, with errors), `umd_workflow.log`

## INPUT / OUTPUT

**`VaspParser.py`** — VASP OUTCAR (also on-the-fly ML runs) → UMD file.

```
VaspParser.py -f OUTCAR -i first_step
-s every_s_steps
```

**out:** `OUTCAR.umd.dat`

**`AbinitParser.py`** — ABINIT MD output (layout of 6.4.1, constant cell) → UMD file.

```
AbinitParser.py -f abinit.out -i first_step
-s every_s_steps
```

**out:** `abinit.out.umd.dat`

**`LAMMPSParser2umd.py`** — LAMMPS dump (+ log) → UMD file; any box, NPT, metal/real units.

```
LAMMPSParser2umd.py -f dump -l log.lammps -u metal
-t timestep_fs -e Si,O -i -s
```

**out:** `<dump>.umd.dat`

**`repair_umd.py`** — repairs a UMD file of an old VaspParser.py (before 2.7) when the OUTCAR is gone.

```
repair_umd.py -f old.umd.dat -o new.umd.dat
```

**out:** `<base>.repaired.umd.dat` (velocities, unwrapped and reduced positions recomputed)

**`check_umd.py`** — 14 quality checks of a run (times, jumps, energy drift, time step, momentum, equilibration, T, stress, core overlap, charges): OK / WARNING.

```
check_umd.py -f run.umd.dat -i skip -s sampling
-r radii.txt -q Fe:3
```

**out:** `<base>.check.dat` (+ `.overlap.dat` with `-r`)

## STRUCTURAL PROPERTIES

**`gofr_umd.py`** — pair distribution functions g(r) and running coordination numbers of all pairs, or of central–adjacent elements; any cell, NPT.

```
gofr_umd.py -f run.umd.dat -i skip -s sampling
-d 0.01 -c O -a Si,Mg
```

**out:** `<base>.gofr.dat` (with -c/-a: `<base>.O-SiMg.gofr.dat`)

**`analyze_gofr.py`** — first maximum, minimum (bond cutoff), coordination number and mean distance of every pair; automatic, `-v` to check and correct.

```
analyze_gofr.py [-f <base>.gofr.dat] -c Si,Mg -a O
-v
```

**out:** `<base>.maximum_bonds.inp` (cutoffs), `.average_bonds.inp`, `.coordinations.inp`, `gofrs_summary.txt`

**`bonding_umd.py`** — the bonds of every snapshot, from per-pair cutoffs or one length; any cell, NPT.

```
bonding_umd.py -f run.umd.dat
-c <base>.maximum_bonds.inp (or -l 2.2) -i skip
-s sampling
```

**out:** `<base>.bonding.dat`

**`speciation_and_angles.py`** — chemical species (r1: central atoms + neighbours; r0: polymers/molecules), angles, bond lifetimes.

```
speciation_and_angles.py -f run.umd.dat
-b <base>.bonding.dat -c Si,Mg -a O -r 1 -t 1 -l 1
-m 1
```

**out:** `<base>.r1.popul.dat`, `.r1.stat.dat`, `.r1.step.dat`, `.angles.dat`, `.bondslives.dat`

**`lifetimes.py`** — plot of the lifetime of every species (columns: central elements; rows: coordination).

```
lifetimes.py -f <base>.r1.popul.dat -t min_life
-x 3,9 -l 12 -m -v
```

**out:** `<base>.r1.lifetimes.pdf` (with `-l`: liquid and vapor figures)

**`liquid_and_vapor_speciation.py`** — splits the species into liquid and vapor from the size of their r0 cluster.

```
liquid_and_vapor_speciation.py
-p <base>.r1.popul.dat -s <base>.r0.step.dat -l 12
```

**out:** `<base>.r1.liquid.popul.dat`, `.liquid.stat.dat`, `.vapor.popul.dat`, `.vapor.stat.dat`

**`stat-concentrate.py`** — gathers the `*.r0.stat.dat` of the runs of a folder (same P, T), with the liquid/vapor partitioning.

```
stat-concentrate.py -l max_vapor_size
-t min_time_fs -c Na,K
```

**out:** one table, its name records the options

## TRANSPORT PROPERTIES

**msd_umd.py** — mean square displacements (FFT, all time origins) and diffusion coefficients with errors; also along axes and the diffusion tensor.

```
msd_umd.py -f run.umd.dat -i skip -s sampling
-m elements|atoms -a cart|cryst|tensor -n 5 -b -e
```

**out:** <base>.msd.dat, .msd.blocks.dat, .diffusion.dat (+ .diffusion.tensor.dat)

**analyze_msd.py** — refits the D of all the MSD files below the folder, with another window; -v: plots, two clicks set the fit window.

```
analyze_msd.py -s start_fs -e end_fs -v
```

**out:** diffusivities.txt per folder (D, error, R², alpha, fit window)

**viscosity_umd.py** — shear viscosity by Green–Kubo from the stress tensor, with block errors and a convergence warning.

```
viscosity_umd.py -f run.umd.dat -i skip -n 4
-e end_steps -c all|offdiag -v
```

**out:** <base>.stress.scf.dat (stress autocorrelations), <base>.visc.dat (running viscosity, η)

**msd_clusters_umd.py** — MSD of the centre of mass of every chemical species of a popul file.

```
msd_clusters_umd.py -f run.umd.dat
-p <base>.r1.popul.dat -b min_life_steps
-c max_size
```

**out:** <base>.r1__<species>.msd.dat, one per species

**analyze_msd_clusters.py** — diffusion coefficients of the species, from the instances living through the fit window; -v: two clicks set it.

```
analyze_msd_clusters.py -s start_fs -e end_fs -v
```

**out:** diffusivities_clusters.txt per folder (check alpha: short-lived species stay ballistic)

## THERMODYNAMIC PROPERTIES

**averages.py** — means and errors of any field; -a: density, P, T, E, heat capacity, with drift warnings.

```
averages.py -f a.umd.dat,b.umd.dat -i skip -a [-p
Pressure] [-l] [-v]
```

**out:** terminal; fullthermo.dat with -a (one line per file)

**vibrations_umd.py** — velocity autocorrelations, vibrational densities of states with error bands, D from the VACF; checks of the temperatures.

```
vibrations_umd.py -f run.umd.dat -i skip -t T
-w window -n 4 -a cart -v
```

**out:** <base>.vels.scf.dat, .vibr.dat, .vibr.err.dat, .vacf.diffusion.dat

**vdos_thermo.py** — harmonic thermodynamics from the vDOS (ZPE, E, F, S, Cv), quantum corrections, 2PT entropy of liquids.

```
vdos_thermo.py -f run.umd.dat -t 300,1000
```

**out:** <base>.vdos.thermo.dat, <base>.2pt.dat

**thermo_integration_umd.py** — free energy and entropy by thermodynamic integration over runs at several λ (VASP SCALEE); ≥ 8 λ.

```
thermo_integration_umd.py
-f l1.umd.dat,l2.umd.dat,... -i skip -t T -v
```

**out:** thermo_integration.dat (+ .pdf with -v)

## BUILDING AND CONVERTING

**build_umd.py** — new structures from a snapshot, a POSCAR or an empty box: supercell, shuffle, insert, scale, density, vacuum, replace, remove; MB velocities.

```
build_umd.py -f run.umd.dat -i -1
--supercell 2,2,2 --insert molecules.dat -t 3000
-o umd,poscar,xyz
```

**out:** <base>[.<snapshot>].build<k>.umd.dat, .POSCAR, .xyz; --extract 0,0-9: <base>.<atoms>.umd.dat

**umd2out.py** — UMD snapshots → extended XYZ, POSCAR, LAMMPS data and dump, DeepMD training set.

```
umd2out.py -f run.umd.dat -i first -s every [-k
snapshot] -r xyz,poscar,lammps,deepmd,lammpstrj -u
```

**out:** <base>.xyz, ._<k>_.POSCAR.vasp, ._<k>_.lammps.data, .deepmd/, .lammpstrj

**umd_clean.py** — removes the outputs of the package, to start afresh; keeps the UMD files, OUTCARs and your own files.

```
umd_clean.py -r -y [--delete]
```

**out:** without -y only the list; -y moves them to umd_cleaned_<date>/, --delete deletes them